

ChipChat Assignment Report

Tanvir Shahriar | LLM4ChipDesign | Notebook: ChipChat_hw.ipynb

1. Setup

- LLM: nvidia/nemotron-3-super-120b-a12b through the NVIDIA API, using the OpenAI client with OPENAI_BASE_URL set once at the top of the notebook. max_tokens was raised from 1024 to 4096 because longer replies were being cut off.
- Verification: Icarus Verilog against the provided testbenches, and Yosys for the CPU cell count.
- Workflow: each example starts with the notebook prompt. When the simulation failed, I sent the error and, where needed, a short explanation back to the LLM as a follow-up message in the same conversation, so it saw its previous design and my feedback.

2. Part I – Eight notebook examples

#	Example	Feedback rounds	Final result
1	Binary to BCD	1	Pass
2	Sequence detector	2	Pass
3	Shift register	1	Fails, testbench issue
4	ABRO state machine	6	Pass
5	Dice roller	1	Pass
6	LFSR	2	Pass
7	Sequence generator	5	Pass
8	Traffic light	1	Pass

Example 1 – Binary to BCD

The LLM produced a correct converter using divide and modulo by 10, but named the ports `bin_in` and `bcd_out`, so the testbench could not connect to it. One round giving the error and the required port names fixed it, and all 32 input values pass.

Example 2 – Sequence detector

The first design used a different module name and a `typedef enum`. After round 1 it compiled but failed at *Cycle 8, Expected: 1, Got: 0*, because `sequence_found` went high one cycle late. Round 2 told it the output must be high in the same cycle as the final `0b101`, and the corrected design passes.

Example 3 – Shift register

The LLM's shift register is correct for the specification. After one round to match the port names it fails with *Test case 0 failed. Expected: 00001010, Got: xxxxxxxx*. The testbench never resets the register before its first check yet expects `00001010`, and its expected values shift every cycle regardless of `shift_enable`, so a correct design cannot pass it. I kept the correct design and noted this in the notebook.

Example 4 – ABRO state machine

This needed the most help, 6 rounds. The LLM's state behaviour was wrong, and fixing one failing test often broke another. It passed only after I gave the exact expected behaviour: A and B high goes to 0100 with O high, only A goes to 0010, only B goes to 0001, and neither holds the state.

Example 5 – Dice roller

The LLM used a 16-bit LFSR as the random source and rolled on the rising edge of roll. It declared variables inside an always block, which is a syntax error, and used different port names. One round fixed both, and all 4000 rolls are in range.

Example 6 – LFSR

Round 1 fixed compile errors and the reset port name. The design then failed because the LLM read “taps at locations 1, 4, 6, 7” as bit numbers starting from 0. Round 2 said the locations start from 1, and it corrected the taps. It passes all 256 cycles.

Example 7 – Sequence generator

5 rounds. The first design used a constant array that Icarus does not support. After that the LLM struggled with timing: 0xAF appeared one cycle late, then appeared twice, and then the LLM went back to its first version. It passed after I asked for the output to come directly from the sequence pointer, plus one last round where it fixed a compile error from the error message alone.

Example 8 – Traffic light

The design was correct except that timeout was declared as a wire but assigned in an always block. One round with the compiler error fixed it, and all four transition checks pass.

3. Part II – CPU example

Starting prompt, from the assignment figure

Let us make a brand new microprocessor design together. We're severely constrained on space and I/O. We have to fit in 1000 standard cells of an ASIC, so I think we will need to restrict ourselves to an accumulator based 8-bit architecture with no multi-byte instructions. Given this, how do you think we should begin?

With reasoning enabled, the model spent all 16,384 completion tokens on reasoning and returned no answer, so the CPU conversation was run with reasoning turned off.

Conversation

LLM proposal. The model suggested an 8-bit instruction with a 3-bit opcode in bits [7:5], a 2-cycle FETCH/EXECUTE control, an accumulator, program counter, stack, status flags, and separate program and data memories. It got stuck on jumps, because a full address did not fit in a one-byte instruction.

Round 1 – architecture decision. I kept its opcode field and FETCH/EXECUTE control, and removed the stack, the extra flags and the immediate instructions to stay within the cell budget. I chose one shared program/data memory so a jump can use the address in the instruction directly, which solved the jump problem. I set the instructions to LDA, STA, ADD, SUB, AND, JMP, JZ and HLT, started with a 32-byte memory, and gave the port names and memory name my testbench needs.

Round 2 – compile error. halted was declared twice. Given the iverilog error, the LLM fixed it. The CPU then passed the test program, but Yosys reported **1601 cells**, 256 of them flip-flops for the 32-byte memory.

Round 3 – area. I gave the Yosys result and asked for a 16-byte memory with a 4-bit program counter and address. The result passes and synthesizes to **838 cells**, under the 1000-cell budget.

Final design

Feature	Choice
Datapath	8-bit accumulator, 4-bit program counter, 8-bit instruction register
Memory	16 × 8 shared program/data memory
Instruction format	[7:5] opcode, [4] unused, [3:0] address, every instruction is one byte
Instructions	000 LDA, 001 STA, 010 ADD, 011 SUB, 100 AND, 101 JMP, 110 JZ, 111 HLT
Control	2-state FSM: FETCH and EXECUTE
Conditional branch	JZ jumps when the accumulator is 0
Verification	Program computing 5×3 by repeated addition: <i>CPU test passed: $5 * 3 = 15$</i>
Area, Yosys	1601 cells with 32 bytes, 838 cells with 16 bytes

4. Part III – Reflection

CPU versus the eight smaller examples

For the small examples the specification was fixed, and feedback mostly meant passing simulator errors back. For the CPU the open-ended prompt gave a long proposal, and the useful work came from me choosing between its ideas and cutting scope to meet the cell budget, while the LLM wrote the Verilog. So human feedback was especially helpful for the more complex design, because it supplied the decisions the model could not make itself. Once the architecture was clear, the CPU needed only 3 rounds, fewer than ABRO or the sequence generator.

How human feedback influenced the design

- Every example and the CPU failed on the first attempt, and all except the shift register, a testbench problem, passed after feedback.
- Compiler errors and wrong names were fixed in one round from the error message alone.
- Timing and behaviour errors needed more than the error message. Short hints were often not enough, and the LLM only succeeded once the expected behaviour was stated explicitly.

Strengths of ChipChat

- Fast from description to working design: three of the eight examples passed after a single feedback round.
- Simulator and synthesis results are easy to pass back, and the LLM acts on them well.

Limitations of ChipChat

- The LLM often used Verilog features the simulator does not support, and used the wrong port names in almost every first attempt.
- It was weak on cycle timing and sometimes went back to an earlier wrong version, which caused most of the multi-round failures.

Examples needing significant correction

ABRO, 6 rounds with the expected behaviour given explicitly, the sequence generator, 5 rounds, and the CPU, where I chose the architecture and the area was reduced from 1601 to 838 cells. The shift register still fails because of the testbench, not the design.